GaiaOps ESG

Cidades ESG Inteligentes

Pipeline de CI/CD, containerização e orquestração
de uma API Java Spring Boot + MongoDB

FIAP · Navegando pelo mundo DevOps

Integrante	RM
João Victor Semente Dias	RM562065

Repositório: https://github.com/JotaDiasDevs/gaiaops-esg

13/10/2026

Sumário

1. O projeto
2. Pipeline de CI/CD
3. Docker: arquitetura, imagem e comandos
4. Testes automatizados
5. Evidências: pipeline em execução
6. Evidências: ambientes de staging e produção
7. Desafios encontrados e como foram resolvidos
8. Tecnologias utilizadas
9. Checklist de entrega

1. O projeto

1.1 Ponto de partida

O GaiaOps é a plataforma de gestão ESG do município fictício de **Vila Nova do Horizonte** (cerca de 90 mil habitantes, quatro distritos). Ele surgiu na Fase 5 como uma base MongoDB com cinco collections: `equipamentos`, `medicoes`, `residuos`, `programas_sociais` e `conformidade`. Elas cobrem os três pilares ESG: telemetria ambiental, coleta de resíduos, programas sociais e governança municipal.

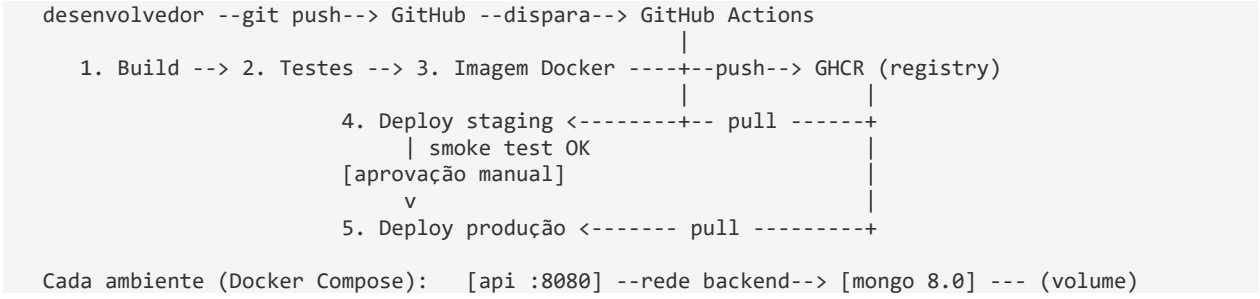
Para esta fase, a base foi transformada em uma **API Java 21 + Spring Boot 3.5** e adaptada para um ambiente de produção simulado. A API reaproveita integralmente o trabalho anterior: os mesmos validadores `$jsonSchema` e índices são aplicados automaticamente no container do banco, e os 100 documentos da Fase 5 entram como carga inicial.

1.2 A aplicação

Recurso	Descrição
`/api/equipamentos`	CRUD completo. Exclusão lógica e atualização com `$set`, que preserva os atributos específicos de cada tipo
`/api/medicoes`	Ingestão de leituras de sensores com **alertas automáticos** (IQAr, ruído, perdas de água, nível de córregos)
`/api/indicadores`	Painel ESG dos três pilares, calculado por agregações no MongoDB
`/api/residuos`, `/api/programas-sociais`, `/api/conformidade`	Consultas, resumo de resíduos e licenças vencidas ou a vencer
`/actuator/health`, `/actuator/info`	Saúde (aplicação + banco) e versão/ambiente/commit implantado
`/` e `/swagger-ui.html`	Página inicial com o painel ao vivo e documentação interativa

1.3 Visão geral da solução

Figura 1 - Fluxo do código até produção



2. Pipeline de CI/CD

2.1 Ferramenta: GitHub Actions

O pipeline foi implementado no **GitHub Actions** (`.github/workflows/ci-cd.yml`). A escolha considerou a integração nativa com o repositório, os **Environments** com aprovação manual e segredos por ambiente, o **GitHub Container Registry (GHCR)** para publicar a imagem e os runners Linux com Docker, que permitem rodar testes com containers e fazer o deploy com Docker Compose sem infraestrutura adicional.

2.2 Etapas

#	Job	O que faz	Quando
1	Build	Compila com Maven (Java 21 Temurin, cache de dependências), empacota o `.jar` e o publica como artefato	push e PR
2	Testes automatizados	`mvn verify`: testes unitários (surefire) e de integração com MongoDB real via Testcontainers (failsafe). Publica relatórios, cobertura JaCoCo e um resumo na página da execução	push e PR
3	Imagem Docker	Build da imagem pelo `Dockerfile` com cache de camadas e push no GHCR com a tag do commit e `latest`	push na main
4	Deploy staging	`deploy.sh staging` (pull da imagem + `docker compose up --wait`), smoke test, print da página, coleta de logs e tag `staging` na imagem	após o 3
5	Deploy produção	Aguarda aprovação manual. Repete o deploy com a mesma imagem e a configuração de produção, roda o smoke test e aplica a tag `production`	após aprovação

2.3 Lógica do pipeline

- **Portões em sequência.** Cada job declara `needs` do anterior. Um teste quebrado não gera imagem, e uma imagem que falha no smoke test de staging nunca chega à produção.
- **Build once, deploy many.** A imagem é construída uma única vez (job 3) e identificada pelo hash do commit. Staging e produção baixam exatamente a mesma imagem. Entre os ambientes muda apenas a configuração (`deploy/staging.env` e `deploy/production.env`).
- **Pull requests** executam apenas build e testes, como validação antes do merge.
- **Aprovação manual.** O job de produção usa o environment `production`, configurado com *Required reviewers*. A execução para e espera um revisor aprovar.
- **Segredos fora do código.** As senhas do MongoDB vêm dos GitHub Secrets de cada environment. Sem eles, o `deploy.sh` gera senhas aleatórias no momento do deploy. Em ambos os casos elas são mascaradas no log (`::add-mask::`).
- **Promoção rastreável.** Após cada deploy bem-sucedido, a imagem recebe a tag do ambiente (`staging` e `production`) no GHCR. A qualquer momento é possível saber qual commit está em cada ambiente, também visível em `/actuator/info`.
- **Mesmos scripts no pipeline e na máquina do desenvolvedor.** O workflow só chama `deploy/deploy.sh` e `deploy/smoke-test.sh`, que também funcionam localmente.

2.4 Smoke test pós-deploy

Após subir cada ambiente, `deploy/smoke-test.sh` faz requisições reais e só deixa o pipeline seguir se todas as verificações passarem:

#	Verificação
1	`/actuator/health/readiness` = UP (aplicação pronta e MongoDB conectado)
2	`/actuator/info` informa o ambiente correto (`staging` ou `producao`)
3	`/actuator/info` informa o commit que está sendo implantado
4	`/api/indicadores` devolve o painel ESG calculado
5	`/api/equipamentos` devolve os dados da carga inicial
6	Cadastro inválido é recusado com HTTP 400 (validação ativa)
7	A página inicial é publicada
8	Configuração do ambiente aplicada: Swagger disponível em staging e **desabilitado em produção**

2.5 Diferenças entre os ambientes

	Staging	Produção
Perfil Spring	`staging`	`prod`
Porta publicada	8081	8082
Swagger UI	habilitado	desabilitado
Detalhes no health check	sim (Mongo, readiness)	não (só UP/DOWN)
Nível de log da aplicação	DEBUG	INFO
Limite de memória da API	768 MB	1 GB
Aprovação para deploy	automática	manual (required reviewers)

2.6 Onde os ambientes rodam

Os jobs de deploy rodam por padrão nos **runners hospedados do GitHub**: máquinas Linux com Docker criadas para cada job e descartadas ao final. O deploy é completo e verificável. A imagem é baixada do registry, os containers sobem com rede, volume e variáveis de ambiente, o smoke test valida o ambiente e as evidências (respostas da API, print da página, estado e logs dos containers) ficam salvas como artefatos da execução. O ambiente, porém, existe apenas durante o job.

> Para ambientes persistentes, o workflow aceita um **self-hosted runner**: basta registrar uma máquina Linux com Docker e criar a variável `DEPLOY_RUNNER`. Os mesmos jobs passam a implantar nessa máquina, onde staging (8081) e produção (8082) permanecem no ar, sem alterar nenhum script.

2.7 Trecho do workflow: job de produção

Listagem 1 - .github/workflows/ci-cd.yml (job 5)

```
  deploy-production:
    name: 5. Deploy producao
    needs: [docker, deploy-staging]
```

```
    runs-on: ${{ vars.DEPLOY_RUNNER || 'ubuntu-latest' }}
    # Com "Required reviewers" configurado no environment "production"
    # (Settings > Environments), o job aguarda aprovacao manual antes de rodar.
    environment:
      name: production
    permissions:
      contents: read
      packages: write
    env:
      AMBIENTE: production
      API_IMAGE: ${{ needs.docker.outputs.imagem }}
      APP_COMMIT: ${{ github.sha }}
    steps:
      - name: Checkout
        uses: actions/checkout@v7

      - name: Login no GitHub Container Registry
        uses: docker/login-action@v4
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

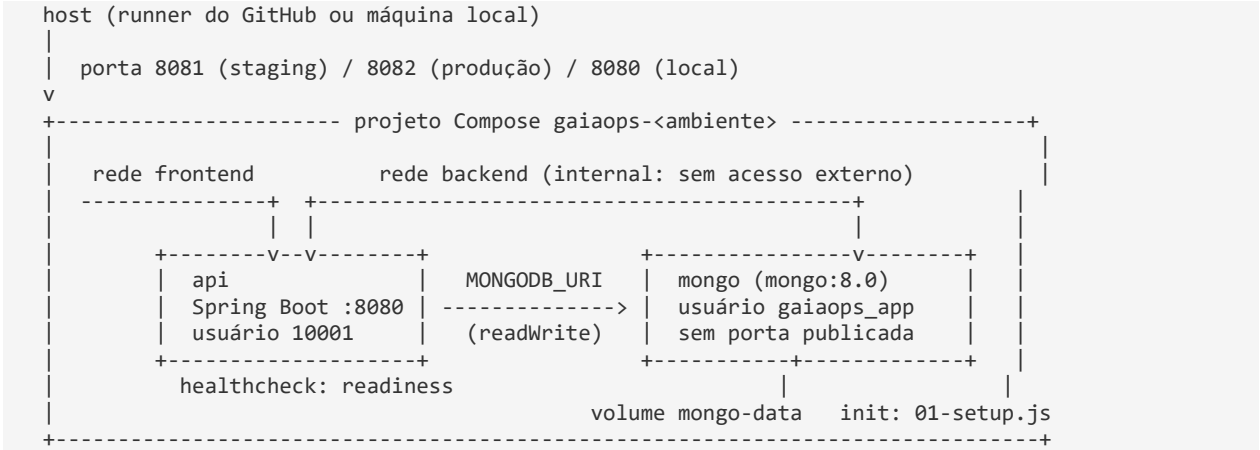
      - name: Deploy com Docker Compose
        env:
          MONGO_ROOT_PASSWORD: ${{ secrets.MONGO_ROOT_PASSWORD }}
          MONGO_APP_PASSWORD: ${{ secrets.MONGO_APP_PASSWORD }}
        run: ./deploy/deploy.sh "$AMBIENTE"

      - name: Smoke test
        run: ./deploy/smoke-test.sh "$AMBIENTE"
```

3. Docker: arquitetura, imagem e comandos

3.1 Arquitetura dos containers

Figura 2 - Containers, redes e volume de um ambiente



- **Dois serviços:** `mongo` e `api`. A API só inicia quando o MongoDB está *healthy* (`depends_on: condition: service_healthy`).
- **Duas redes:** `frontend`, onde a API publica a porta, e `backend`, marcada como `internal`, que liga só a API ao banco. O MongoDB **não expõe porta** para fora do Docker.
- **Volume nomeado** `mongo-data`: os dados sobrevivem a reinícios e a novos deploys.
- **Script de inicialização** `mongo/init/01-setup.js`: roda uma única vez, na criação do volume, e cria as collections com os validadores, os índices da Fase 5 e um usuário da aplicação com permissão só no banco dela (menor privilégio).
- **Variáveis de ambiente:** tudo que muda entre ambientes vem de `.env` (local) ou `deploy/<ambiente>.env`. Senhas obrigatórias usam `${VAR:?}`, então o Compose se recusa a subir sem elas.

3.2 Dockerfile

Listagem 2 - Dockerfile (sem os comentários)

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /workspace

COPY pom.xml .
RUN mvn -B -ntp dependency:go-offline

COPY src ./src
RUN mvn -B -ntp package -DskipTests

FROM eclipse-temurin:21-jre-alpine AS runtime

LABEL org.opencontainers.image.title="GaiaOps ESG API" \
      org.opencontainers.image.description="API de gestao ESG para Cidades Inteligentes"

RUN addgroup -S -g 10001 gaiaops && adduser -S -u 10001 -G gaiaops gaiaops

WORKDIR /app
COPY --from=build --chown=10001:10001 /workspace/target/gaiaops-esg.jar app.jar

USER 10001:10001
EXPOSE 8080

ENV JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=75 -XX:+ExitOnOutOfMemoryError"
```

```
HEALTHCHECK --interval=10s --timeout=5s --start-period=40s --retries=5 \
  CMD ["sh", "-c", "wget -q -O /dev/null http://localhost:8080/actuator/health/readiness || exit
1"]

ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Estratégia	Benefício
Multi-stage build	Imagem final só com JRE + `.jar`. Maven, JDK e código-fonte ficam no estágio de build
`pom.xml` antes do código	Dependências em camada própria, reaproveitada do cache quando só o código muda
Base `eclipse-temurin:21-jre-alpine`	Runtime mínimo e mantido, com menor superfície de ataque
Usuário sem privilégios (UID 10001)	O processo não roda como root no container
`JAVA_TOOL_OPTIONS`	Heap proporcional ao limite de memória do container; encerra em OutOfMemoryError
`HEALTHCHECK` no readiness	*healthy* só com a API pronta e o MongoDB respondendo, base do `compose up --wait`
`ENTRYPOINT` em formato exec	A JVM é o PID 1 e recebe o SIGTERM, garantindo o encerramento gracioso
`.dockerignore`	Só `pom.xml` e `src/` vão para o build; `.env`, `.git` e `target` ficam fora

3.3 docker-compose.yml

Listagem 3 - docker-compose.yml (sem os comentários)

```
services:

  mongo:
    image: mongo:8.0
    restart: unless-stopped
    environment:
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_ROOT_USER:?defina MONGO_ROOT_USER}
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_ROOT_PASSWORD:?defina MONGO_ROOT_PASSWORD}
      MONGO_INITDB_DATABASE: ${MONGO_DATABASE:-gaiaops_esg}
      MONGO_APP_USER: ${MONGO_APP_USER:?defina MONGO_APP_USER}
      MONGO_APP_PASSWORD: ${MONGO_APP_PASSWORD:?defina MONGO_APP_PASSWORD}
    volumes:
      - mongo-data:/data/db
      - ./mongo/init:/docker-entrypoint-initdb.d:ro
    networks:
      - backend
    healthcheck:
      test: ["CMD", "mongosh", "--quiet", "--eval", "db.adminCommand('ping').ok"]
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 20s

  api:
    image: ${API_IMAGE:-gaiaops-esg-api:local}
    build:
      context: .
    restart: unless-stopped
    depends_on:
      mongo:
        condition: service_healthy
    environment:
      SPRING_PROFILES_ACTIVE: ${SPRING_PROFILES_ACTIVE:-local}
      GAIAOPS_AMBIENTE: ${GAIAOPS_AMBIENTE:-local}
      GAIAOPS_SEED_ENABLED: ${GAIAOPS_SEED_ENABLED:-true}
```

```
      APP_COMMIT: ${APP_COMMIT:-local}
      MONGODB_URI: mongodb://${MONGO_APP_USER}:${MONGO_APP_PASSWORD}@mongo:27017/$
{MONGO_DATABASE:-gaiaops_esg}?authSource=${MONGO_DATABASE:-gaiaops_esg}
    ports:
      - "${APP_PORT:-8080}:8080"
    networks:
      - frontend
      - backend
    deploy:
      resources:
        limits:
          memory: ${API_MEMORY_LIMIT:-768m}

volumes:
  mongo-data:

networks:
  frontend:
  backend:
    internal: true
```

3.4 Imagem criada

Item	Valor
Registry	GitHub Container Registry (GHCR)
Nome	`ghcr.io/<usuario>/gaiaops-esg`
Tags	hash completo do commit (imutável), `latest`, `staging` e `production`
Base	`eclipse-temurin:21-jre-alpine`
Conteúdo	JRE 21 + `app.jar` (Spring Boot), usuário 10001, porta 8080, healthcheck
Metadados	labels OCI (origem, revisão, data de criação) gerados pelo `docker/metadata-action`

3.5 Comandos utilizados

Comando	Finalidade
`docker compose up -d --build`	Constrói a imagem e sobe API + MongoDB localmente
`docker compose ps`	Estado e healthcheck dos serviços
`docker compose logs -f api`	Acompanha o log da API
`docker compose down` / `down -v`	Para o ambiente (mantendo ou apagando o volume)
`./deploy/deploy.sh staging|production`	Sobe um ambiente: `docker compose -p gaiaops-<amb> --env-file deploy/<amb>.env up -d --wait`
`./deploy/smoke-test.sh staging|production`	Valida o ambiente implantado (8 verificações)
`docker compose pull api`	No pipeline: baixa do GHCR a imagem já testada
`docker buildx imagetools create --tag ...:staging`	Promove a imagem (aplica a tag do ambiente)
`./mvnw verify`	Build + testes unitários e de integração

4. Testes automatizados

Os testes rodam no job 2, com `./mvnw verify`, e qualquer falha interrompe o pipeline.

Classe	Tipo	O que verifica	Testes
`RegrasDeAlertaTest`	Unitário	Regras de alerta automático: IQAr, ruído diurno/noturno, perdas de água, cotas de córrego e leituras incompletas (esquema flexível)	9
`EquipamentoControllerTest`	Unitário (camada web)	Validação de entrada, códigos HTTP (201, 204, 400, 404, 409), formato Problem Details e conversão snake_case	7
`GaiaOpsApplicationIT`	Integração	Aplicação completa + **MongoDB 8.0 real (Testcontainers)**: carga inicial, painel ESG, ciclo CRUD com exclusão lógica, alerta automático, regra de negócio (422) e readiness	6

A cobertura de código é medida pelo JaCoCo e publicada no artefato `relatorios-de-teste`. O resumo com aprovados, falhas e cobertura aparece na página de cada execução do pipeline (print 03).

5. Evidências: pipeline em execução

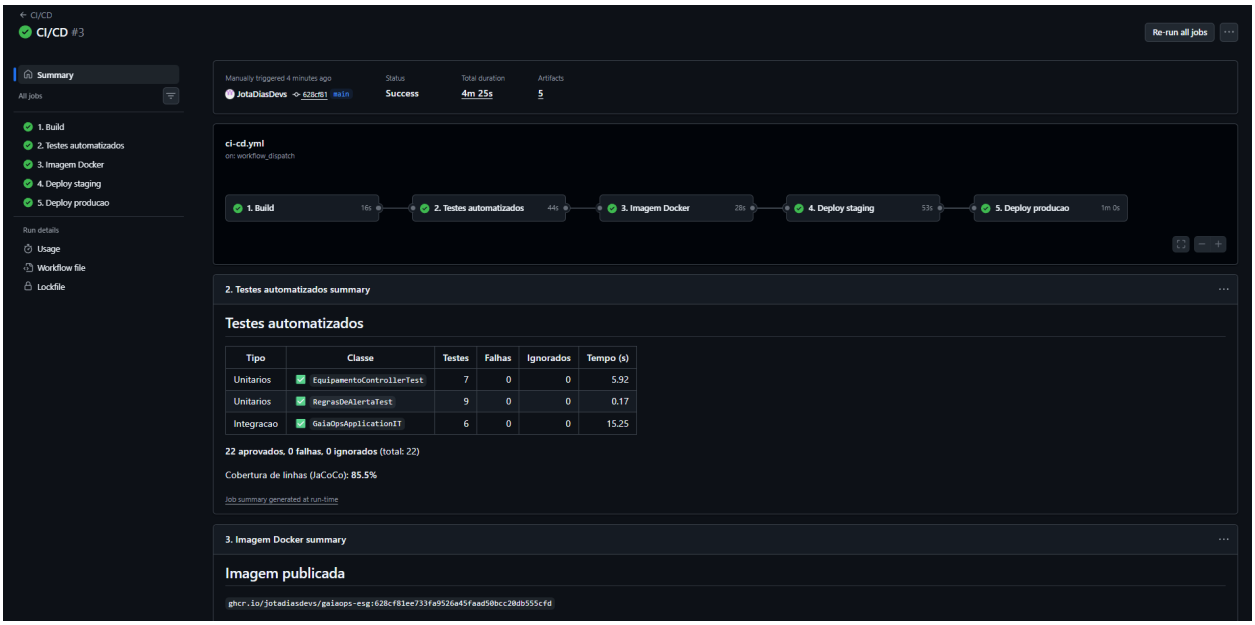


Figura 3 - Execução completa do pipeline (5 jobs)

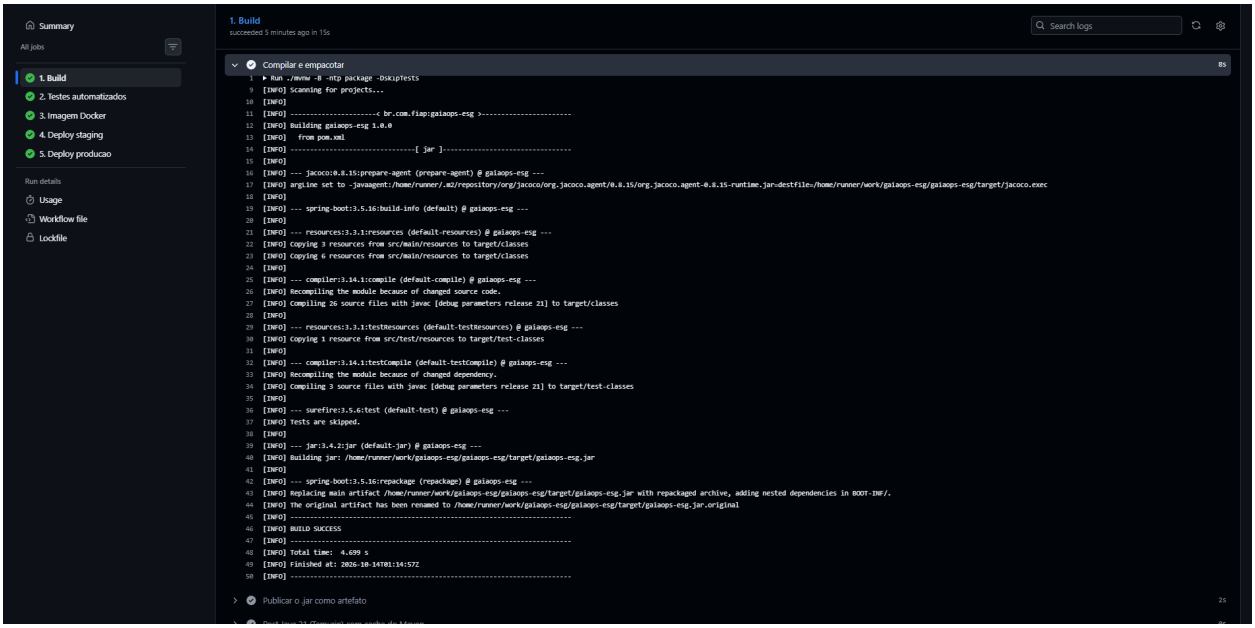


Figura 4 - Job de build



Figura 5 - Resultado dos testes automatizados

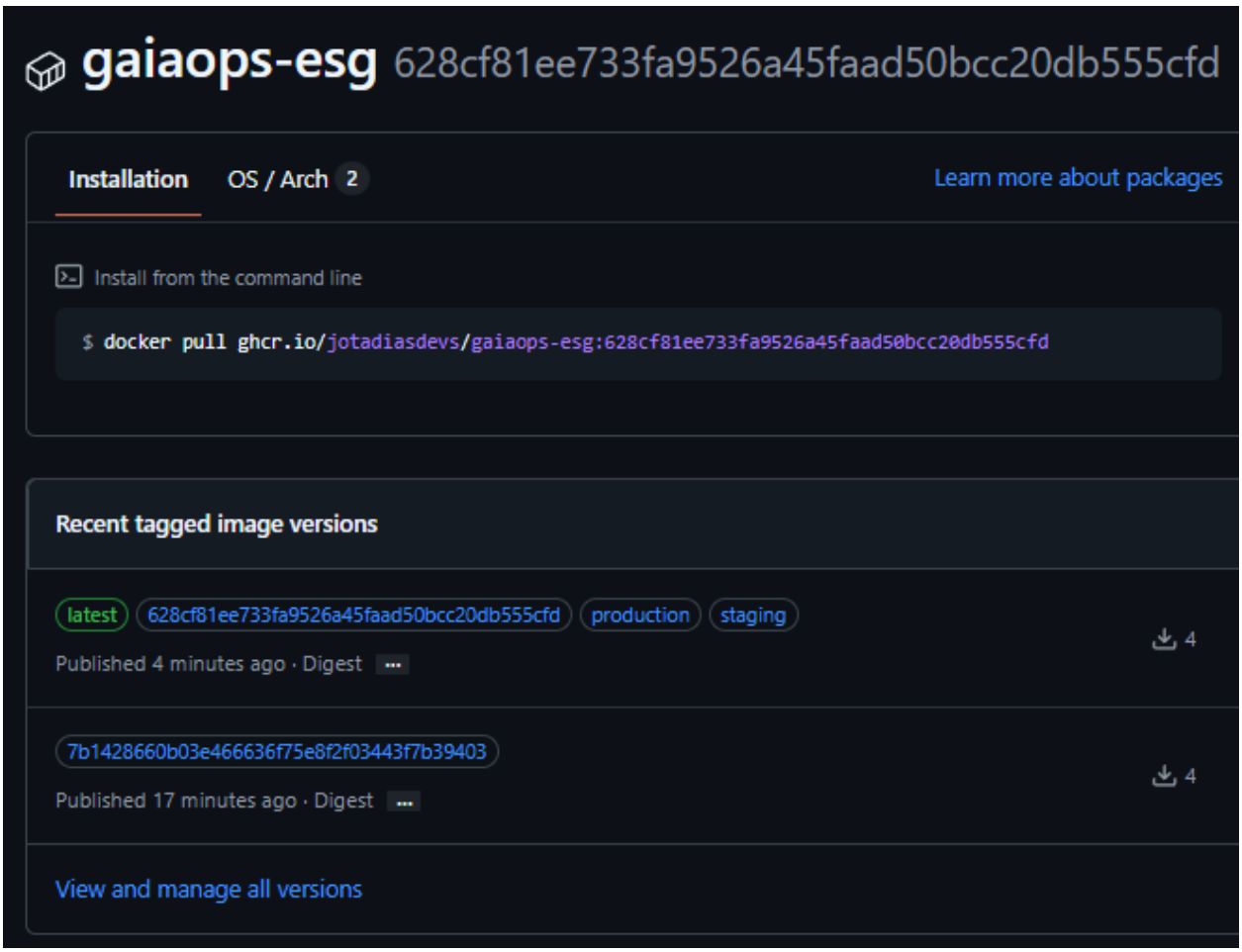


Figura 6 - Imagem publicada no GHCR

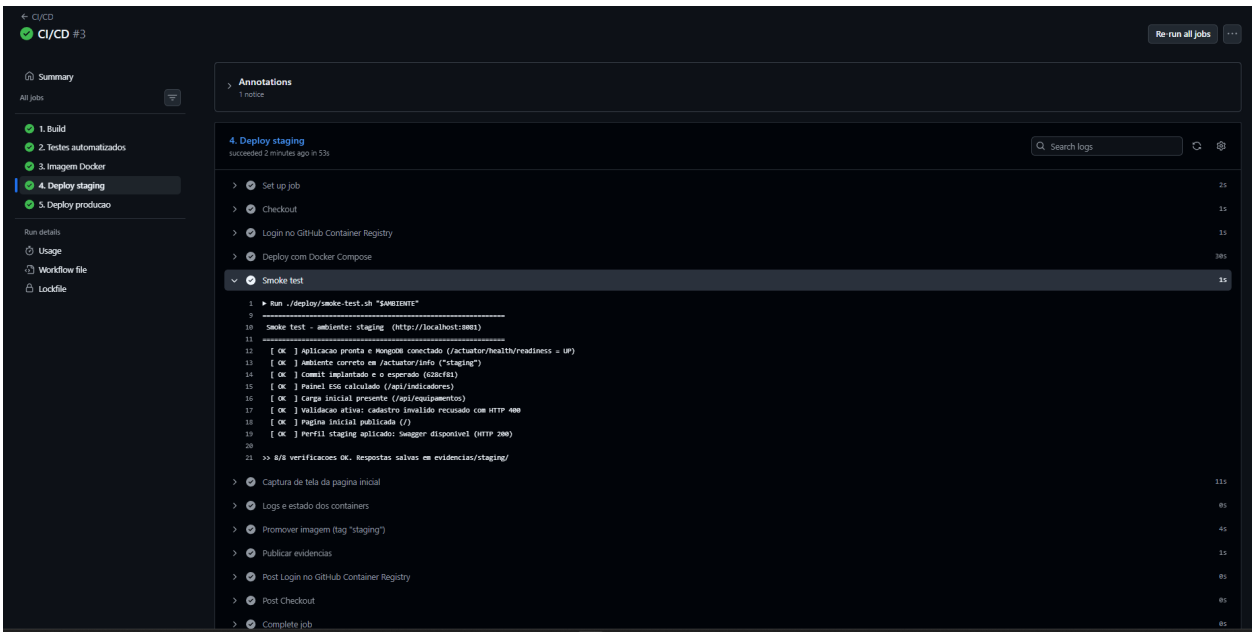


Figura 7 - Deploy em staging e smoke test

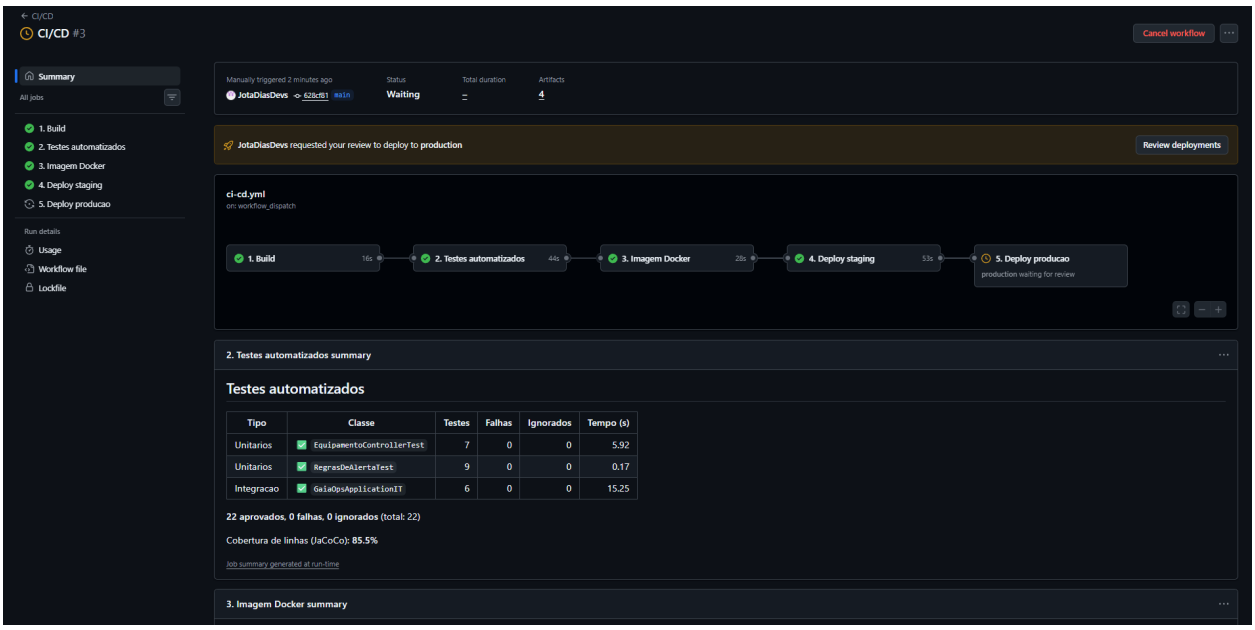


Figura 8 - Aprovação manual para produção

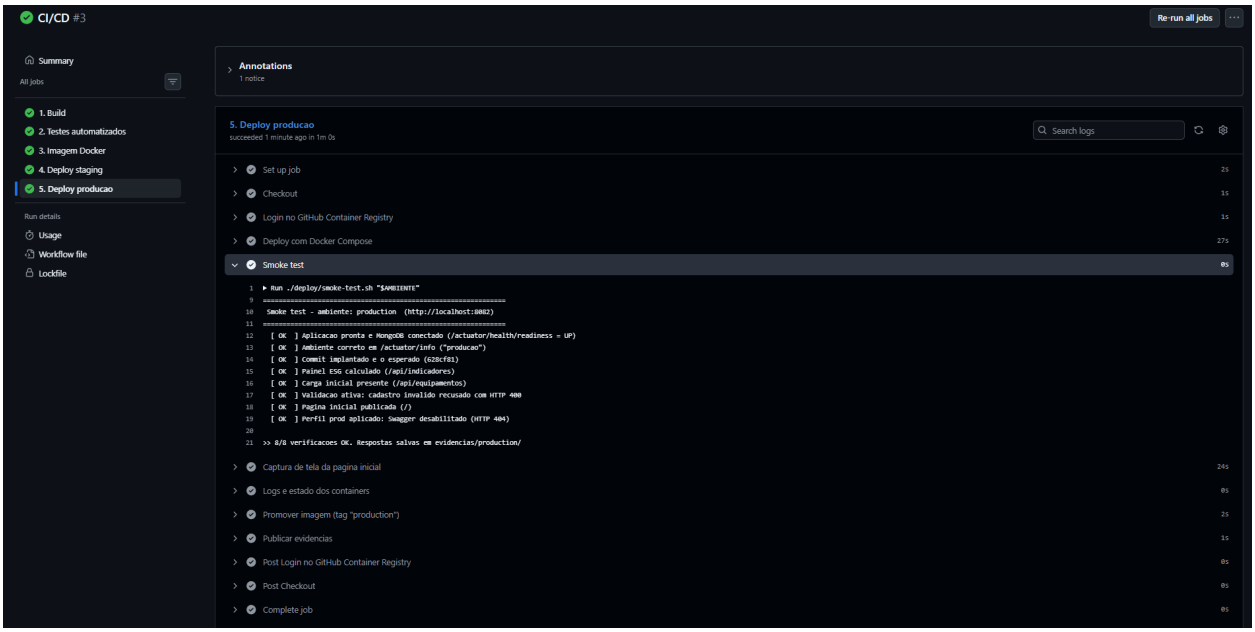


Figura 9 - Deploy em produção e smoke test

6. Evidências: ambientes de staging e produção

As capturas abaixo são geradas pelo próprio pipeline (`deploy/captura-tela.sh`, Chrome headless) logo após o smoke test de cada ambiente e publicadas nos artefatos `evidencias-staging` e `evidencias-production`. A faixa colorida no topo da página identifica o ambiente; os chips mostram a versão, o commit implantado e a saúde da aplicação.

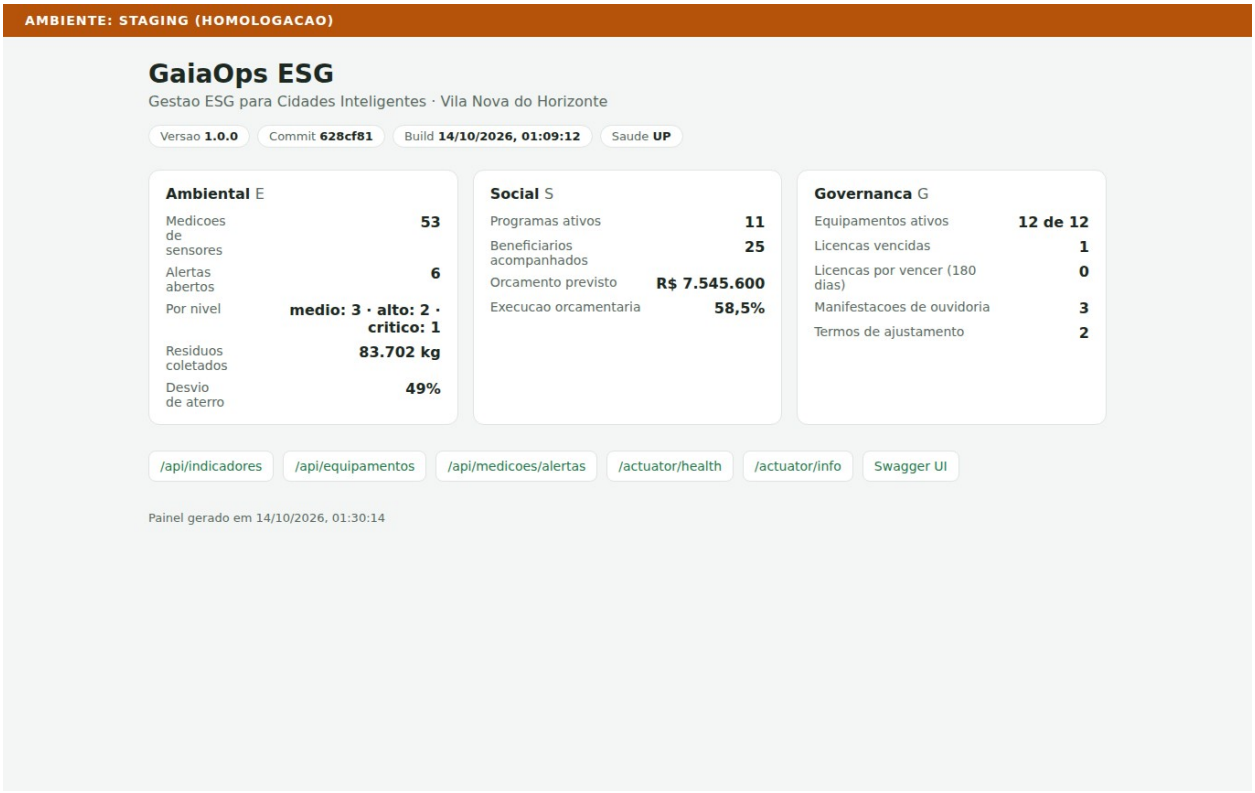


Figura 10 - Ambiente de staging funcionando

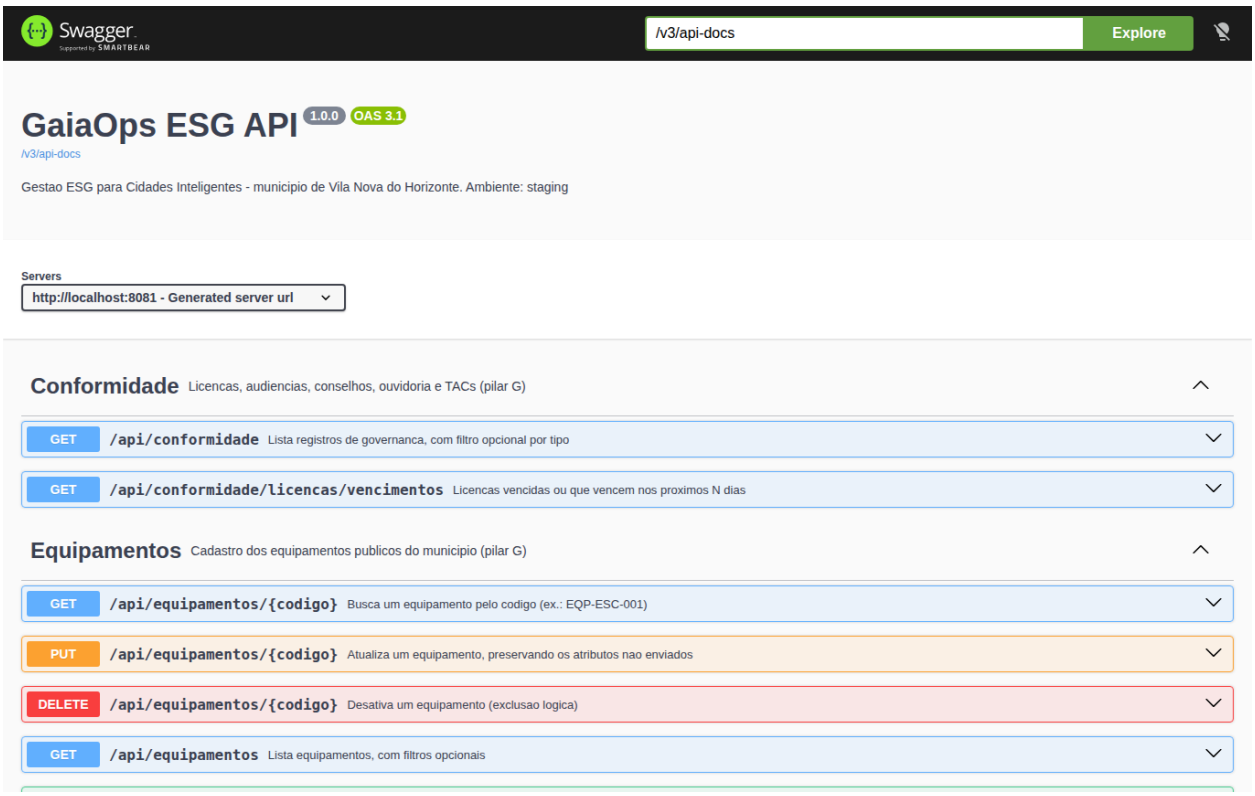


Figura 11 - Swagger UI disponível em staging

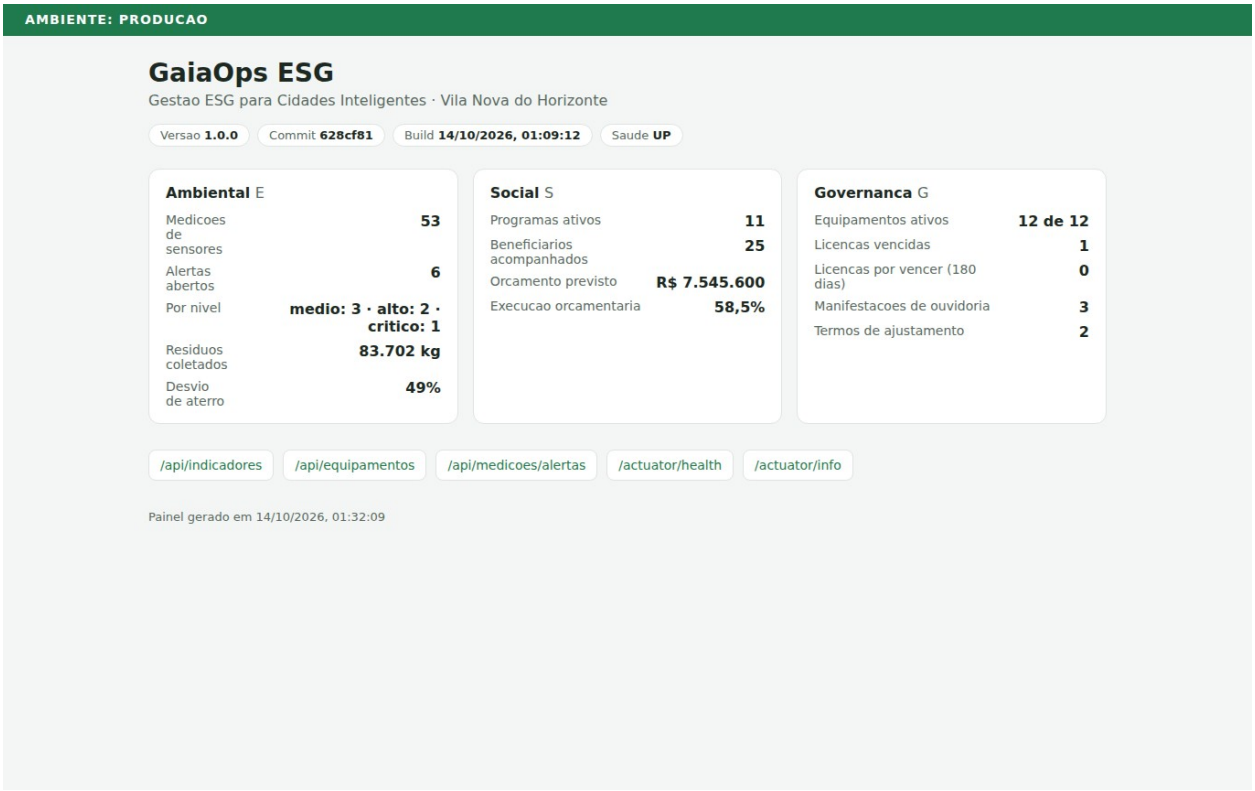


Figura 12 - Ambiente de produção funcionando

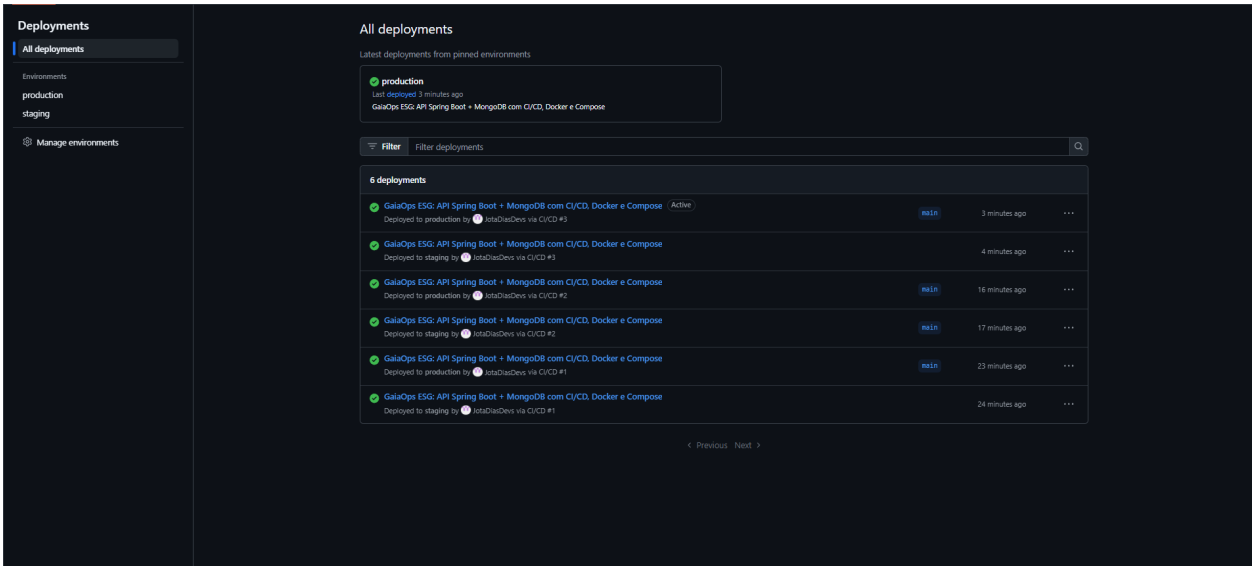


Figura 13 - Deployments por ambiente no GitHub

7. Desafios encontrados e como foram resolvidos

7.1 Validar Docker e pipeline sem Docker na máquina de desenvolvimento

Problema. A estação de desenvolvimento não tinha Docker instalado, então não dava para subir os containers localmente durante a construção.

Solução. Validação em camadas: `docker compose config` (binário oficial, que não precisa do daemon) para conferir o Compose em cada ambiente; `hadolint` para o Dockerfile; `actionlint` e `shellcheck` para o workflow e os scripts; o mesmo `mongo/init/01-setup.js` executado no MongoDB local; e a API rodando contra ele com os perfis staging e prod, aprovada pelo mesmo smoke test do pipeline. Os testes de integração usam `@Testcontainers(disabledWithoutDocker = true)`: são ignorados sem Docker e sempre executados no pipeline.

7.2 Certificado TLS recusado pela JVM

Problema. O Maven falhava com `PKIX path building failed` ao baixar dependências: a rede inspeciona o tráfego TLS com um certificado que o Java não reconhece (o `curl`, que usa o repositório do Windows, funcionava).

Solução. Apontar a JVM para o repositório de certificados do Windows (`-Djavax.net.ssl.trustStoreType=Windows-ROOT` em `MAVEN_OPTS`), só na máquina local. O projeto não precisa de nenhuma alteração por causa disso.

7.3 Preservar o esquema flexível da Fase 5 na API

Problema. Mapear os documentos para classes fixas faria um PUT apagar os campos específicos de cada tipo de equipamento, como os turnos de uma escola ou a vazão de uma ETE.

Solução. A API trabalha com `org.bson.Document`. A atualização usa `$set` campo a campo, e os campos controlados não podem ser sobrescritos via atributos livres. O teste de integração comprova que um atributo não reenviado continua no documento.

7.4 Erros de validação saindo no formato padrão do Spring

Problema. O tratador de Problem Details do Spring Boot tinha precedência sobre o da aplicação, e as mensagens de validação por campo se perdiam. Dois testes da camada web acusaram o problema.

Solução. O tratador da aplicação passou a estender `ResponseEntityExceptionHandler`. O Spring Boot então desativa o dele, e todos os erros (inclusive JSON malformado) saem no mesmo formato, com mensagens em português.

7.5 Nome de campo inesperado no JSON

Problema. Com a estratégia SNAKE_CASE do Jackson, o campo `licencasAVencer` virou `licencas_avencer`, e a página inicial não encontrava o valor. O teste manual da API revelou o problema.

Solução. O campo foi renomeado para `licencasPorVencer` (`licencas_por_vencer`), e o teste de integração passou a verificar o nome.

7.6 Senhas do banco sem versionar segredos

Problema. O deploy precisa de senhas, mas elas não podem ficar no repositório nem aparecer nos logs.

Solução. Os arquivos `deploy/*.env` só têm configuração não sensível. As senhas vêm dos GitHub Secrets do environment ou são geradas no deploy; localmente, ficam em um arquivo ignorado pelo Git. Todas são mascaradas no log, e o Compose recusa subir sem elas (`${VAR:?}`).

7.7 Docker Engine 29 nos runners e a API do Testcontainers

Problema. Versões recentes do Docker Engine recusam clientes com API antiga, o que pode quebrar o Testcontainers.

Solução. Spring Boot 3.5.16 (Testcontainers 1.21.4) e `src/test/resources/docker-java.properties` fixando a API mínima em 1.44.

7.8 Projeto editado no Windows, executado no Linux

Problema. Scripts com fim de linha CRLF ou sem permissão de execução quebram no runner Linux.

Solução. `.gitattributes` força LF em `.sh`, `mvnw`, `.yml` e `.env`, e os scripts são versionados com bit de execução (`git update-index --chmod=+x`).

7.9 Primeira execução chegou à produção sem aprovação

Problema. Os environments `staging` e `production` não existiam antes do primeiro push. O GitHub os criou automaticamente, sem regras de proteção, e a execução #1 implantou em produção sem esperar revisão.

Solução. Configuração de *Required reviewers* no environment `production` e nova execução manual (`workflow_dispatch`). A partir dela, o pipeline para em *Waiting for review* antes de produção (Figura 8). Também ficou registrado que o *Re-run* de uma execução repete o mesmo commit: para implantar um commit novo, é preciso disparar uma nova execução.

7.10 Garantir que produção receba exatamente o que foi testado

Problema. Reconstruir a imagem a cada deploy poderia gerar artefatos diferentes entre staging e produção.

Solução. Build once, deploy many: a imagem é construída uma vez, identificada pelo hash do commit, e promovida por tags. O smoke test confere o commit em `/actuator/info` em cada ambiente.

8. Tecnologias utilizadas

Camada	Tecnologias
Linguagem e framework	Java 21, Spring Boot 3.5 (Web, Validation, Data MongoDB, Actuator), springdoc-openapi
Banco de dados	MongoDB 8.0 com validadores `$jsonSchema` e índices parciais, sparse, multikey, 2dsphere e de texto
Build	Maven 3.9 (Maven Wrapper)
Testes	JUnit 5, Spring MockMvc, Mockito, AssertJ, Testcontainers, JaCoCo
Containers	Docker (multi-stage), Docker Compose
CI/CD	GitHub Actions, GitHub Environments, GitHub Secrets, GitHub Container Registry
Scripts e qualidade	Bash, actionlint, shellcheck, hadolint
Documentação	Markdown, Mermaid, python-docx, LibreOffice

9. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	☑
Dockerfile funcional	☑
docker-compose.yml ou arquivos Kubernetes	☑
Pipeline com etapas de build, teste e deploy	☑
README.md com instruções e prints	☑
Documentação técnica com evidências (PDF ou PPT)	☑
Deploy realizado nos ambientes staging e produção	☑